

Sign-to-Speech (S2S)

Final Documentation

Real-Time AI-Powered Sign Language Interpreter and Translator

Group Name: S2S

Team Members:

Daniel Dousuah

Enos Kimanya

Sudik Dhakal

Shared Account: dominicaugustine65@gmail.com

1 Project Overview

The Sign-to-Speech (S2S) project was developed to bridge the communication gap between the Deaf and Hard-of-Hearing (DHH) community and the hearing world. This project focused on creating a deep learning model capable of translating American Sign Language (ASL) into text and speech. Utilizing MediaPipe as the primary feature extraction backbone, the model tracks and interprets human spatial coordinates across a diverse video dataset to accurately recognize individual signs.

Project Resources:

- GitHub Repository: [Click Here](#)
- WLASL Top 100 Training Videos: [Click Here](#)
- WLASL Annotated Metadata: [Click Here](#)
- WLASL Preprocessed Data: [Click Here](#)

2 Significance of Outcome

The development of a functional S2S system offers multifaceted benefits:

- **Accessibility:** It fosters independence for signers in diverse public and private environments. Real-time conversion facilitates fluid conversations by mitigating the communication delays inherent in traditional text-based or interpreter-reliant methods.
- **Inclusion:** It actively promotes a more inclusive society by leveraging technology to dismantle long-standing language barriers.

3 Literature Review and Existing Works

The Word-Level American Sign Language (WLASL) dataset is currently the largest public resource for ASL research, featuring over 21,000 videos and a 2,000-word vocabulary. While previous deep learning models have performed well on smaller subsets of this data, researchers emphasize that scaling up to the full 2,000-word vocabulary remains a significant challenge due to the subtle physical and spatial nuances between many signs.

In a more targeted application, a 2020 study by Ojha et al. introduced a real-time desktop translator for ASL finger-spelling. Utilizing a webcam and a Convolutional Neural Network (CNN), the system successfully translated hand gestures into text and audio with 95% accuracy. However, this system relies heavily on optimal lighting conditions and is limited to spelling individual letters rather than interpreting full words or phrases.

Furthermore, a 2023 survey by Liang et al. distinguished between basic Sign Language Recognition (identifying isolated signs) and advanced Sign Language Translation (converting consecutive signs into fluent sentences). The study highlighted that the scarcity of high-quality, annotated video data is the primary obstacle to advancing translation systems. Consequently, researchers are increasingly exploring AI-generated synthetic data to supplement model training.

Recent studies have explored various techniques for foundational sign language recognition. Sundar and Bagyammal (2022) utilized MediaPipe for ASL alphabet recognition, highlighting dynamic gestural fluidity but lacking full sentence translation capabilities. Expanding on visual recognition, Kasapbsi et al. (2022) used a CNN to evaluate signs under varied lighting and distances. While this highlights how environmental variables impact accuracy, the study lacks a speech synthesis component to measure end-to-end translation latency.

Alternative methodologies have focused on hardware and cloud deployment. Abraham and Rohini (2018) achieved high ground-truth accuracy using wearable flex sensors to capture gestures, though the

required hardware introduces usability friction and systemic lag. Conversely, Akin-Ponnle (2021) successfully demonstrated a cloud-based CNN for remote digit classification, though its narrow focus on numbers falls short of conversational vocabulary.

Finally, broader methodological surveys by Tao et al. (2024) and Subburaj and Murugavalli (2022) chronicle the evolution from traditional models to modern deep learning and feature extraction techniques. However, both reviews are predominantly classification-centric; they bypass the complexities of deployable, end-to-end sign-to-speech systems and modern transformer architectures.

4 Preliminary Phase

The project involved training a deep learning model from scratch using Python and TensorFlow. The technical workflow was structured as follows:

4.1 Phase 1: Data Acquisition

The WLASL dataset was utilized as the primary data source. From its 21,083 videos encompassing 2,000 distinct words and over 100 signers, we targeted the top 100 most frequently occurring instances, along with their corresponding JSON metadata files. This extraction provided 1,600 robust instances for training and validation. Each data entry consisted of:

- **WLASL_video**: The raw .mp4 video file.
- **WLASL_v0.3.json**: The corresponding metadata file containing:
 - **label**: An integer (0–99) corresponding to a specific gloss word (e.g., "book").
 - **bbox**: Bounding box coordinates identifying the signer within the frame.
 - **frame_start** / **frame_end**: Video frame indices denoting where the sign occurs.
 - **split**: The pre-assigned data subset (train or val).

4.2 Phase 2: Augmentation and Preprocessing

- **Augmentation**: To enhance dataset robustness, we applied a custom Python script to the videos, performing operations such as 10% zoom, horizontal flipping, speed reduction, 10-degree rotation, and brightness adjustments. Each augmented video was synchronized with a newly generated JSON metadata file. This achieved a 1:5 ratio of original to augmented videos, successfully expanding the dataset from 1,600 to approximately 7,000 instances.
- **Preprocessing**: MediaPipe Holistic was employed as the primary feature extractor to capture spatial coordinates of the hands, face, and pose from raw video frames. This process converted 3D visual data into 1D numerical arrays containing 363 specific landmarks per frame, using the signer's nose as the central anchor point. Sequences containing fewer frames than the established threshold were padded with '-99' to ensure uniform tensor shapes. The final input shape passed to the model was (None, 363).

5 Model Architecture

We engineered a custom **Sequential Neural Network** in TensorFlow optimized for 1D temporal data:

- **Masking Layer**: Identifies and masks '-99' padded frames, ensuring downstream layers ignore them during training to prevent data skew.
- **Dense Layer**: Analyzes the relationships among the 363 spatial coordinates within individual frames, expanding them into 256 high-level features.

- **Bidirectional LSTM Layers:** Two Bidirectional Long Short-Term Memory (Bi-LSTM) layers featuring ReLU activation and 128 neurons each were integrated. These layers analyze frame sequences simultaneously in forward and reverse directions, capturing comprehensive temporal context. This dual-perspective approach enabled the model to understand preceding and succeeding movements for more accurate classification. Bi-LSTM also offered superior computational efficiency compared to Conv3D architectures.
- **Classification Layer:** A dense softmax output layer designed to predict the final classification label (ranging from 0 to 99).

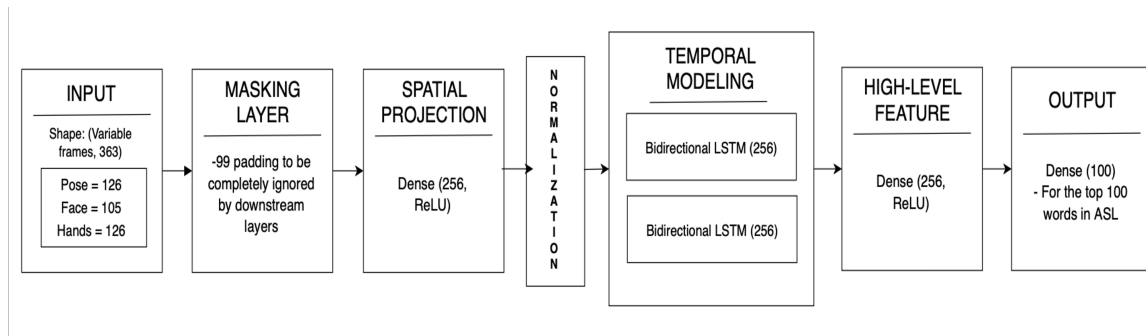


Figure 1: Model Architecture Diagram

6 Training

The model was trained on the augmented dataset of over 7,000 instances. We utilized a batch size of 16, configured the Adam optimizer with a learning rate of $1e-4$, and trained the model over 100 epochs, strictly targeting accuracy improvements.

7 Results

The training cycle proved highly computationally efficient. Individual epochs required an average of 30 to 40 seconds, resulting in a cumulative training time of 60 to 90 minutes.

During the initial phase, both training and validation accuracy surged rapidly, accompanied by sharp declines in loss metrics. While mid-training metrics exhibited slight divergence—indicating mild overfitting where the model memorized training data faster than it generalized—extended training across the full 100 epochs allowed the weights to stabilize. Ultimately, the model achieved a commendable average validation accuracy of approximately 85%, with the validation loss stabilizing just below 0.1.

8 Project Management

8.1 Work Breakdown Structure (WBS)

1. **Research & Data Acquisition** (Weeks 4–5)
2. **Data Augmentation & MediaPipe Integration** (Weeks 6–7)
3. **Model Development** (Weeks 8–9)
4. **Initial Training Phase** (Weeks 10–11)
5. **Training and Fine-Tuning** (Weeks 12–13)
6. **Poster Presentation & Final Submission** (Week 14)

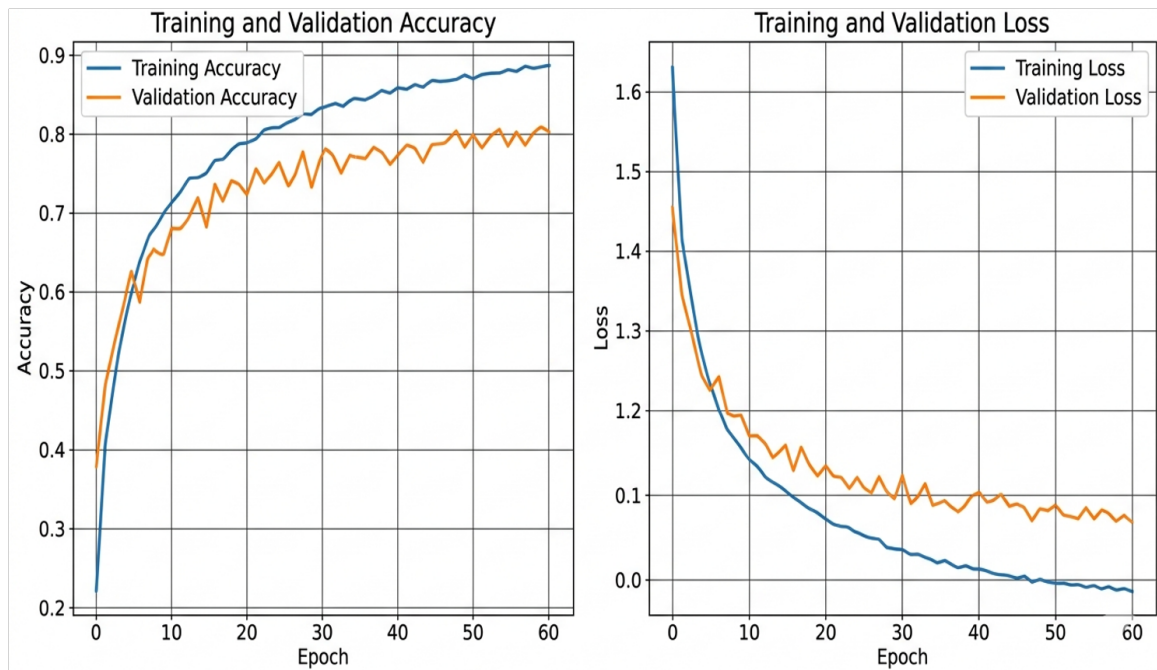


Figure 2: Training & Validation Accuracy Trend

9 Project Timeline (Gantt Chart)

Task / Week	4	5	6	7	8	9	10	11	12	13	14
Research & Data Acquisition	X	X									
Data Augmentation & Preprocessing			X	X							
Model Development					X	X					
Initial Training Phase							X	X			
Fine-Tuning & Evaluation									X	X	
Final Submission & Presentation											X

Table 1: Project Timeline (Feb 2026 – May 2026)

10 Future Directions

10.1 Fine-Tuning and Testing

Future iterations could incorporate transformer architectures or advanced convolutional layers to elevate the validation accuracy beyond 90%. Furthermore, given the limited volume of publicly available annotated datasets, real-world testing was constrained. Subsequent phases of this project should explore using unstructured YouTube videos as unseen test data to evaluate the model's true real-world generalization.

10.2 Speech-to-Sign Integration

While the primary objective of S2S was sign-to-speech translation, a fully holistic communication system requires a reverse "Speech-to-Sign" functionality. This feature was considered during brainstorming but deferred for the following reasons:

- **Generative Complexity:** Unlike recognition models which classify inputs, reverse translation requires a generative engine (e.g., a 3D avatar) to synthesize realistic hand movements, relying heavily on complex Inverse Kinematic (IK) solvers.
- **Time Constraints:** To guarantee high accuracy for the recognition module within our strict 14-week timeline, the team decided to focus exclusively on discriminative recognition tasks.

Future scalability efforts for S2S will prioritize integrating a 3D avatar module to provide real-time visual sign feedback for spoken responses.

Works Cited

1. Abraham, S., and Rohini, V. "Real time conversion of sign language to speech and prediction of gestures using Artificial Neural Network." *Procedia Computer Science*, vol. 143, 2018, pp. 587-594, <https://doi.org/10.1016/j.procs.2018.10.435>.
2. Kasapbaşı, A., Eltayeb, A., et al. "DeepASLR: A CNN based human computer interface for American Sign Language recognition for hearing-impaired individuals." *Computer Methods and Programs in Biomedicine Update*, vol. 2, 2022, 100048, <https://doi.org/10.1016/j.cmpbup.2021.100048>.
3. Akin-Ponnle, A. E. "Cloud-based human sign language digit classification using CNN: A case study of king's-center Akure Nigeria." *Language*, vol. 7, no. 5, 2021.
4. Al-Qurishi, K., and Souissi, M. "Deep Learning for Sign Language Recognition: Current Techniques, Benchmarks, and Open Issues." *IEEE Access*, vol. 9, 2021, pp. 126917-126951, doi: 10.1109/ACCESS.2021.3110912.
5. Baumgärtner, L., Jauss, S., Maucher, J., and Zimmermann, G. "Automated sign language translation: The role of artificial intelligence now and in the future." *Proceedings of the 4th International Conference on Computer-Human Interaction Research and Applications*, 2020.
6. De Sisto, M., et al. "Challenges with Sign Language Datasets for Sign Language Recognition and Translation." *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, European Language Resources Association, 2022, pp. 2478–2487.
7. Khan, A., et al. "Deep Learning Approaches for Continuous Sign Language Recognition: A Comprehensive Review." *IEEE Access*, vol. 13, 2025, pp. 55524-55544, doi: 10.1109/ACCESS.2025.3554046.
8. Li, D., Opazo, C. R., Yu, X., and Li, H. "Word-level Deep Sign Language Recognition from Video: A New Large-scale Dataset and Methods Comparison." *arXiv*, 2020, <https://arxiv.org/abs/1910.11006>.
9. Liang, Z., Li, H., and Chai, J. "Sign Language Translation: A Survey of Approaches and Techniques." *Electronics*, vol. 12, no. 12, 2023, p. 2678, <https://doi.org/10.3390/electronics12122678>.
10. Natarajan, B., et al. "Development of an End-to-End Deep Learning Framework for Sign Language Recognition, Translation, and Video Generation." *IEEE Access*, vol. 10, 2022, pp. 104358-104374.
11. Ojha, A., et al. "Sign Language to Text and Speech Translation in Real Time Using Convolutional Neural Network." *International Journal of Engineering Research & Technology (IJERT)*, vol. 8, no. 15, 2020, pp. 191-196.
12. Patil, U., et al. "Sign Language Translator Using Deep Learning." *EasyChair*, no. 2541, 2020.
13. Subburaj, S., and Murugavalli, S. "Survey on sign language recognition in context of vision-based and deep learning." *Measurement: Sensors*, vol. 23, 2022, 100385, <https://doi.org/10.1016/j.measen.2022.100385>.
14. Sundar, B. "American Sign Language Recognition for Alphabets Using MediaPipe and LSTM." *Procedia Computer Science*, vol. 215, 2022, pp. 642-651, <https://doi.org/10.1016/j.procs.2022.12.066>.
15. Tao, Z., Liu, Y., and Zhu, X. "Sign Language Recognition: A Comprehensive Review of Traditional and Deep Learning Approaches, Datasets, and Challenges." *IEEE Access*, vol. 12, 2024, pp. 75034-75060, doi: 10.1109/ACCESS.2024.3398806.
16. Wong, R., Camgoz, N. C., and Bowden, R. "Sign2GPT: Leveraging Large Language Models for Gloss-Free Sign Language Translation." *arXiv*, 2024, <https://arxiv.org/abs/2405.04164>.